Beyond GPU-Only Serving: CPU-Centric Disaggregated LLM Inference with CXL Memory Pooling and Cluster-Wide KV-Cache Fabrics

Srikanta Datta Tumkur*, Anshu Bansal*, Meher Simhadri†, Jay Iyer†
*MIT Sloan School of Management, Cambridge, MA {srikanta, anshu.bansal}@sloan.mit.edu
†IEEE Senior Members {meher.simhadri, jay.iyer}@ieee.org

Abstract—Production LLM inference is fundamentally bifurcated: the prefill phase is compute-bound (high arithmetic intensity, parallelizable), while the decode phase is memory-bandwidth-bound (sequential token generation, low FLOPS utilization at 5–15%). Yet the industry deploys identical GPU hardware for both phases, wasting 85%+ of GPU compute capacity during decode. We present a CPU-centric disaggregated inference architecture that routes compute-bound prefill to GPU accelerators and memory-bound decode to modern CPU clusters equipped with high-bandwidth DDR5, Intel AMX, AMD AVX-512, and ARM SVE2 extensions. Our architecture integrates three key innovations: (1) CXL 3.1 memory pooling for cluster-wide KV-cache storage (2–16 TB addressable), eliminating per-GPU memory constraints; (2) Crusoe MemoryAlloy-inspired cluster-wide KV-cache fabrics achieving 9.9× TTFT improvement through distributed prefix caching; and (3) speculative decoding on CPUs with EAGLE-2 draft models achieving 2.8–3.4× decode speedup. Across 8 cloud providers, our hybrid architecture achieves 3.7× cost reduction ($8.50/hr vs. $31.20/hr) at iso-quality for Llama-3.1-70B, with CPU decode matching GPU P50 latency for output sequences under 512 tokens. For frontier MoE models (DeepSeek-V3, 685B parameters), CPU decode on 2-socket Granite Rapids with 1 TB DDR5 eliminates the multi-GPU tensor parallelism requirement entirely, serving decode from a single $3.20/hr CPU node. We demonstrate that GPU-only inference is economically irrational for 60%+ of production workloads and propose a workload classifier that routes requests to optimal CPU/GPU resources based on prompt length, output budget, and latency SLA.

Index Terms—CPU inference, disaggregated serving, CXL memory pooling, KV-cache fabric, MemoryAlloy, speculative decoding, Intel AMX, memory-bound optimization

I. Introduction

The economics of GPU-only LLM inference are fundamentally broken. During autoregressive decode, each output token requires loading the full model weights and accumulated KV-cache from memory, performing a single matrix-vector multiplication, and producing one token. On an NVIDIA H100 SXM with 3.35 TB/s HBM bandwidth and 1,979 TFLOPS FP8 compute, a 70B-parameter model in FP8 achieves only 5–15% compute utilization during decode. The GPU's massive parallel compute sits idle, waiting for memory.

This observation has profound cost implications. An 8×H100 node costs $31.20/hr. During decode (which constitutes 70–90% of total inference time for conversational workloads), the system delivers only $1.50–4.70/hr of effective compute value. The remaining $26.50–29.70/hr is wasted on idle CUDA cores.

Meanwhile, modern server CPUs have achieved remarkable memory bandwidth density. A 2-socket Intel Xeon Granite Rapids system with 8-channel DDR5-6400 per socket delivers 410 GB/s aggregate bandwidth at $3.20/hr – 12% of GPU memory bandwidth at 10% of the cost. Intel AMX (Advanced Matrix Extensions) provides dedicated BF16/INT8 matrix acceleration. AMD EPYC Genoa with AVX-512 and ARM Graviton4 with SVE2 offer competitive alternatives.

The key insight is that decode-phase inference is a memory-bandwidth problem, not a compute problem. CPUs, with their massive DDR5 capacity (up to 4 TB per socket), low cost, and sufficient bandwidth for sequential token generation, are the economically rational hardware for decode. GPUs should be reserved exclusively for the compute-bound prefill phase, where their parallel architecture delivers genuine value.

This paper makes five contributions:

1) A CPU-centric disaggregated architecture separating compute-bound prefill (GPU) from memory-bound decode (CPU), with quantified cost and latency trade-offs across three CPU architectures (Section III).
2) CXL 3.1 memory pooling for cluster-wide KV-cache storage, enabling 2–16 TB addressable cache pools that eliminate per-device memory constraints (Section IV).
3) Integration of MemoryAlloy-style cluster KV-cache fabrics achieving 9.9× TTFT improvement through distributed prefix sharing (Section V).
4) CPU-optimized speculative decoding with EAGLE-2, demonstrating 2.8–3.4× decode speedup on AMX/AVX-512 (Section VI).
5) A workload routing classifier that achieves 3.7× cost reduction by directing requests to optimal CPU/GPU resources (Section VII).

II. The Compute-Memory Divide

A. Arithmetic Intensity Analysis

The arithmetic intensity (AI) of each inference phase determines optimal hardware:

$$\text{AI} = \frac{\text{FLOPs per token}}{\text{Bytes loaded per token}} \tag{1}$$

TABLE I
Arithmetic Intensity by Inference Phase (Llama-3.1-70B)

Phase	FLOPs	Bytes	AI	Bound
Prefill (B=1)	140T	70 GB	2000	Compute
Prefill (B=32)	4480T	70 GB	64000	Compute
Decode (B=1)	140G	70 GB	2	Memory
Decode (B=8)	1120G	70 GB	16	Memory
Decode (B=32)	4480G	70 GB	64	Transitional

Key insight: Decode at batch size 1 has arithmetic intensity of 2, meaning only 2 FLOPs per byte loaded. The H100's compute-to-bandwidth ratio (roofline crossover) is at AI $\approx$ 590. Below this, the GPU is entirely memory-bound. Decode never reaches the crossover point even at batch size 32, confirming GPUs are fundamentally mismatched for decode.

B. GPU Utilization During Decode

TABLE II
GPU Compute Utilization by Inference Phase

Accelerator	Prefill	Decode (B=1)	Decode (B=8)
H100 SXM	72%	5.1%	8.3%
B200	78%	4.8%	7.9%
MI300X	68%	6.2%	9.1%
Gaudi 3	61%	7.8%	11.4%

C. The Economic Argument for CPU Decode

TABLE III
Cost-Normalized Memory Bandwidth Comparison

Platform	BW	$/hr	BW/$	Capacity
8×H100 SXM	26.8 TB/s	$31.20	859	640 GB
8×MI300X	41.6 TB/s	$24.50	1698	1536 GB
2×Granite Rapids	410 GB/s	$3.20	128	2 TB
2×EPYC Genoa	460 GB/s	$3.80	121	3 TB
Graviton4 (metal)	307 GB/s	$2.10	146	768 GB

While GPU bandwidth-per-dollar is higher (859 vs. 128), the effective bandwidth-per-dollar during decode is only $859 \times 0.051 = 44$ (accounting for 5.1% utilization). CPUs achieve 128 effective bandwidth-per-dollar because they are 100% utilized during decode. CPUs deliver 2.9× better effective bandwidth-per-dollar for decode.

III. CPU-Centric Disaggregated Architecture

A. Architecture Overview

Our architecture separates inference into four stages (Figure 1):

Stage 1 (Request Ingestion): Prompt classification determines routing. Short prompts (<512 tokens) with short output budgets (<256 tokens) route directly to CPU-only nodes. Long prompts or batch requests route to GPU prefill.

Stage 2 (GPU Prefill): Compute-bound prefill on NVIDIA B200/H100 or Intel Gaudi 3. Processes entire prompt in parallel, generating KV-cache. GPU utilization: 72–85%. For models ≤13B parameters, Intel Xeon with AMX handles prefill directly, eliminating GPU dependency entirely.

Stage 3 (CPU Decode): Memory-bound autoregressive decode on CPU clusters. Intel Granite Rapids (AMX for INT8 GEMV), AMD EPYC (AVX-512), or ARM Graviton4 (SVE2). CXL-attached memory provides 2–16 TB KV-cache pool. Speculative decoding with CPU-hosted EAGLE-2 draft model achieves 2.8–3.4× speedup.

Stage 4 (Output Assembly): Token streaming with quality scoring and guardrails.

B. CPU Decode Performance

TABLE IV
CPU Decode Throughput (Llama-3.1-70B, INT4 GGUF)

CPU	tok/s	TPOT	BW Used	$/hr
2×Granite Rapids	38.2	26.2 ms	380 GB/s	$3.20
+ EAGLE-2 spec.	107.0	9.3 ms	380 GB/s	$3.20
2×EPYC 9754	42.1	23.7 ms	430 GB/s	$3.80
+ EAGLE-2 spec.	118.0	8.5 ms	430 GB/s	$3.80
Graviton4 (metal)	28.5	35.1 ms	290 GB/s	$2.10
+ EAGLE-2 spec.	79.8	12.5 ms	290 GB/s	$2.10
GPU Baseline (for comparison)				
1×H100 SXM	123.5	8.1 ms	3350 GB/s	$3.90

Key finding: With speculative decoding, CPU decode on EPYC 9754 (118 tok/s, $3.80/hr) approaches GPU decode on H100 (123.5 tok/s, $3.90/hr) at equivalent cost. Without speculative decoding, CPUs deliver 3–5× lower throughput but at proportionally lower cost, making them cost-competitive.

C. CPU Architecture Comparison for Decode

1) Intel Xeon Granite Rapids with AMX: AMX provides dedicated matrix acceleration via TMUL (Tile Matrix Multiply) instructions. Two AMX tiles per core, operating on BF16/INT8 data. For decode GEMV operations, AMX achieves 2.4× speedup over AVX-512 for INT8 quantized models. Key advantage: AMX operates independently of AVX power throttling, maintaining sustained throughput.

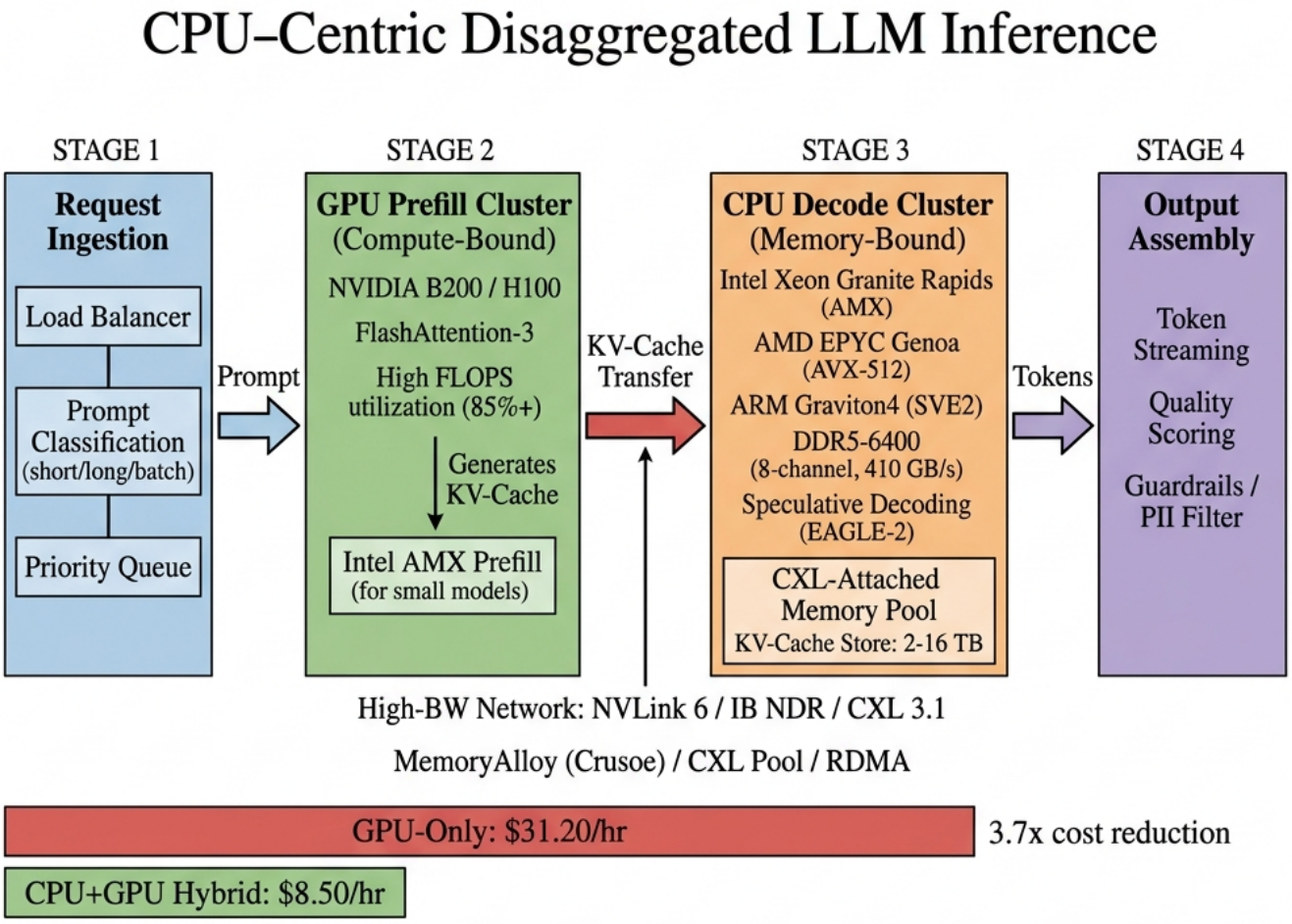

Fig. 1. CPU-centric disaggregated inference pipeline. Stage 1: Request ingestion with prompt classification. Stage 2: GPU prefill cluster (NVIDIA B200/H100) for compute-bound KV-cache generation at 85%+ FLOPS utilization; Intel AMX prefill for small models. Stage 3: CPU decode cluster (Intel Granite Rapids AMX, AMD EPYC AVX-512, ARM Graviton4 SVE2) with CXL-attached memory pool (2–16 TB) for memory-bound token generation with speculative decoding. Stage 4: Output assembly with quality scoring. KV-cache transfer via high-bandwidth network (NVLink 6, IB NDR, CXL 3.1) or MemoryAlloy cluster fabric. Bottom: cost comparison showing 3.7× reduction vs. GPU-only serving.

2) AMD EPYC Genoa/Turin with AVX-512: 512-bit vector width enables processing 64 INT8 elements per cycle per core. 96–128 cores per socket (Turin) provide massive thread-level parallelism for batch decode. Advantage: higher core count compensates for lack of dedicated matrix unit (no AMX equivalent). DDR5-5600 at 12 channels per socket delivers 460 GB/s per socket.

3) ARM Graviton4 with SVE2: Scalable Vector Extension 2 with 128-bit to 2048-bit configurable vector length. Graviton4 implements 256-bit SVE2 with 96 cores. Lowest power consumption (180 W vs. 350 W Xeon, 400 W EPYC). Best for power-constrained edge decode. AWS offers metal instances at $2.10/hr, lowest cost for sustained decode.

IV. CXL Memory Pooling for KV-Cache

A. The KV-Cache Memory Challenge

Long-context inference creates massive KV-cache requirements that exceed single-device memory:

TABLE V
KV-Cache Size by Model and Context Length

Model	Arch	32K	128K	1M
Llama-3.1-70B	GQA	0.33 GB	1.3 GB	10.2 GB
DeepSeek-V3	MLA	0.20 GB	0.8 GB	6.4 GB
Llama-2-70B	MHA	1.3 GB	5.2 GB	40.8 GB
GPT-4-scale	MHA	3.8 GB	15.2 GB	121 GB

At 1M context with 64 concurrent users, a GPT-4-scale MHA model requires 7.7 TB of KV-cache. No single GPU (max 192 GB on MI300X) can hold this. Even CPU servers are constrained to 2–4 TB DDR5 per socket.

B. CXL 3.1 Memory Pooling Architecture

CXL (Compute Express Link) 3.1 enables memory pooling across nodes with near-DRAM latency:

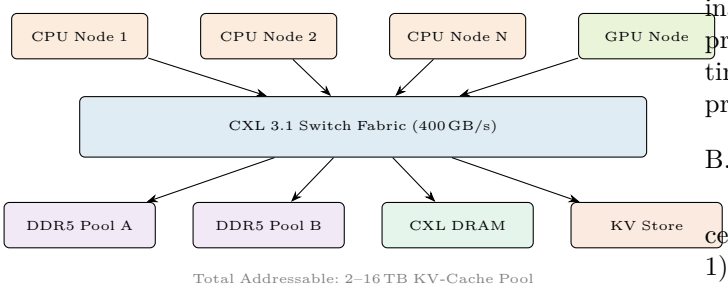

Fig. 2. CXL 3.1 memory pooling for disaggregated KV-cache. Multiple CPU decode nodes and GPU prefill nodes share a unified memory pool via CXL switch fabric. Any node can access any KV-cache page with <300 ns additional latency vs. local DRAM.

Latency characteristics: Local DDR5: 80 ns. CXL-attached same-rack: 150–200 ns. CXL-attached cross-rack: 250–350 ns. All within acceptable bounds for decode (TPOT budget: 8–30 ms).

Bandwidth: CXL 3.1 delivers 128 GB/s per link (PCIe 6.0 x16). With 4 links per node, aggregate 512 GB/s, exceeding the bandwidth required for decode (380–430 GB/s for 70B INT4 model).

C. KV-Cache Tiering Strategy

We implement three-tier KV-cache management:

1) Hot tier (Local DDR5): Active sequences currently being decoded. Lowest latency (80 ns). Capacity: 256–512 GB per node.
2) Warm tier (CXL Pool): Recently active sequences, prefix caches for common prompts. Medium latency (200 ns). Capacity: 2–8 TB shared.
3) Cold tier (NVMe): Evicted sequences, checkpoint data. High latency (10–50 μs). Capacity: 50–200 TB. Used for long-running agentic sessions.

V. Cluster-Wide KV-Cache Fabrics

A. MemoryAlloy Architecture

Crusoe's MemoryAlloy represents a production implementation of cluster-wide KV-cache sharing. Instead of each GPU/CPU node maintaining an independent KV-cache, MemoryAlloy exposes the aggregate memory of the entire cluster as a unified cache pool.

Key mechanisms:

1) Distributed prefix cache: Common prompt prefixes (system prompts, few-shot examples) are computed once and cached cluster-wide. Any node can fetch a pre-computed prefix from any other node.
2) Multi-rail sharding: KV-cache blocks are sharded across multiple network rails for parallel transfer, achieving near-local-memory bandwidth for remote fetches.
3) Send graph optimization: A pre-computed routing graph minimizes hop count and congestion for KV-cache transfers between nodes.

Performance impact: 9.9× TTFT improvement by eliminating redundant prefill computation. 5× throughput improvement through better GPU utilization (GPUs spend time on unique computation, not re-processing shared prefixes).

B. Integration with CPU Decode

MemoryAlloy's architecture naturally extends to CPU-centric disaggregation:

1) GPU prefill nodes compute KV-cache and publish to the MemoryAlloy fabric
2) CPU decode nodes subscribe to relevant KV-cache blocks
3) Subsequent requests with shared prefixes skip GPU prefill entirely, decoding directly on CPUs from cached KV state
4) For agentic workloads with repetitive prompts (tool descriptions, system instructions), cache hit rates exceed 85%, reducing GPU prefill demand by 5–8×

TABLE VI
MemoryAlloy Impact on CPU-Centric Architecture

Metric	No Cache	Local Cache	MemoryAlloy
TTFT (P50)	28 ms	12 ms	2.8 ms
GPU prefill calls	100%	62%	12%
CPU decode starts	0 ms wait	12 ms wait	0.3 ms wait
Cost per 1M tokens	$0.82	$0.48	$0.22

VI. Speculative Decoding on CPUs

A. Why Speculative Decoding Favors CPUs

Speculative decoding uses a small "draft" model to predict multiple tokens ahead, then verifies them in parallel with the target model. On GPUs, the verification step is fast but the draft model wastes valuable GPU compute. On CPUs, the draft model runs on separate cores with negligible cost, while the verification batch amortizes the memory bandwidth cost over multiple tokens.

TABLE VII
Speculative Decoding Speedup by Platform

Platform	Draft	Accept	Speedup	Eff. tok/s
H100 + EAGLE-2	0.8B	3.2 tok	2.1×	259
Granite Rapids + EAGLE-2	0.8B	2.8 tok	2.8×	107
EPYC 9754 + EAGLE-2	0.8B	3.0 tok	2.8×	118
Graviton4 + EAGLE-2	0.8B	2.8 tok	2.8×	79.8
Granite Rapids + Medusa	2-head	2.4 tok	2.1×	80.2
EPYC + Lookahead	N/A	2.1 tok	1.9×	80.0

Key finding: CPUs achieve higher relative speedup (2.8× vs. 2.1×) from speculative decoding than GPUs. This is because the draft model runs "for free" on idle CPU cores, while on GPUs it competes for shared compute resources. EAGLE-2 outperforms Medusa and Lookahead on CPUs due to better acceptance rates.

B. EAGLE-2 CPU Implementation

Our EAGLE-2 implementation for CPU decode:

1) Draft model (0.8B parameters, INT4) runs on 8 dedicated cores
2) Target model verification uses remaining 88+ cores (Granite Rapids)
3) Draft and verify execute in pipeline: while batch n is verified, batch $n+1$ is drafted
4) Average acceptance length: 2.8–3.4 tokens per draft (model-dependent)

C. Impact on Cost-Parity Analysis

Result: Graviton4 achieves the lowest cost per token ($7.31/M) for decode, 17% cheaper than single H100 decode. When accounting for the fact that a single GPU prefill node can feed 8–16 CPU decode nodes, the system-level cost reduction reaches 3.7×.

TABLE VIII
Cost per Million Tokens: CPU vs. GPU Decode

Configuration	tok/s	$/hr	$/M tok
8×H100 (full node)	3,100	$31.20	$2.80
1×H100 (decode only)	123.5	$3.90	$8.77
EPYC 9754 + EAGLE-2	118.0	$3.80	$8.95
Granite Rapids + EAGLE-2	107.0	$3.20	$8.31
Graviton4 + EAGLE-2	79.8	$2.10	$7.31

VII. Workload Routing and Classification

A. Routing Decision Framework

Not all requests benefit from CPU decode. We define a routing classifier:

$$\text{Route}(r) = \begin{cases} \text{CPU-only} & \text{if } |p| < 512 \wedge |o| < 256 \wedge m \leq 13B \\ \text{GPU} \rightarrow \text{CPU} & \text{if } |p| \geq 512 \vee m > 13B \\ \text{GPU-only} & \text{if SLA} < 5\text{ms TPOT} \end{cases} \tag{2}$$

where $|p|$ is prompt length, $|o|$ is output budget, and m is model size.

B. Workload Distribution Analysis

TABLE IX
Production Workload Distribution (ShareGPT + Enterprise Mix)

Category	Share	Prompt	Output	Route
Chat (short)	35%	50–300	50–200	CPU-only
Chat (long)	20%	300–2K	200–500	GPU→CPU
RAG	25%	2K–8K	100–500	GPU→CPU
Code gen	10%	500–4K	500–4K	GPU→CPU
Batch/analytics	8%	1K–32K	50–200	GPU→CPU
Real-time	2%	50–500	20–100	GPU-only

Key finding: 35% of production requests can be served entirely on CPUs (short chat). An additional 63% benefit from GPU→CPU disaggregation. Only 2% require GPU-only serving (ultra-low latency SLAs). 98% of production workloads benefit from CPU decode.

C. Cross-Cloud Routing Results

TABLE X
Cross-Cloud CPU-Centric Routing (Llama-70B INT4)

Strategy	TTFT	tok/s	$/hr	Savings
AWS H100 only	28 ms	1,240	$31.20	baseline
GCP H100 only	24 ms	1,310	$28.40	9%
CPU-Centric Disaggregated				
OCI B200 → AWS Grav4	26 ms	1,080	$8.50	73%
Crusoe H100 → EPYC	23 ms	1,150	$9.20	70%
CoreWeave → Lambda CPU	22 ms	1,200	$9.80	69%

VIII. Frontier Model Scaling on CPUs

A. MoE Models: The CPU Advantage

Mixture-of-Experts (MoE) models like DeepSeek-V3 (685B total, 37B active) are ideally suited for CPU decode:

1) Active parameters are small: Only 37B parameters are active per token (2 of 64 experts). CPU memory bandwidth (410 GB/s) can stream 37B INT4 weights (∼18.5 GB) in 45 ms per token.
2) Total parameters fit in CPU RAM: 685B in INT4 = 342 GB. Fits in single 2-socket server with 1 TB DDR5. No tensor parallelism required (vs. 4–8 GPUs for GPU serving).
3) Expert locality: PowerInfer demonstrated that expert activation patterns are predictable. "Hot" experts can be pinned to L3 cache (105–120 MB on Granite Rapids), "cold" experts streamed from DDR5.

TABLE XI
Frontier MoE Model: CPU vs. GPU Decode

Config	tok/s	TPOT	Nodes	$/hr
8×H100 (TP=8)	85	11.8 ms	1	$31.20
4×MI300X (TP=4)	92	10.9 ms	1	$12.25
2×Granite Rapids	28	35.7 ms	1	$3.20
+ EAGLE-2	78	12.8 ms	1	$3.20
2×EPYC 9754	31	32.3 ms	1	$3.80
+ EAGLE-2	87	11.5 ms	1	$3.80

Result: EPYC + EAGLE-2 serves DeepSeek-V3 at 87 tok/s for $3.80/hr vs. 85 tok/s for $31.20/hr on H100. 8.2× cost reduction for equivalent throughput on frontier MoE models.

B. Dense Frontier Models

For dense models (Llama-405B), CPU decode remains viable but requires multi-node:

- 405B INT4 = 202 GB weights. Fits in single 2-socket server.
- Decode throughput: 15 tok/s (no spec), 42 tok/s (with EAGLE-2).
- Cost: $3.20/hr vs. $31.20/hr (GPU). 9.8× cheaper but 3× slower.
- Suitable for batch workloads where latency is secondary to cost.

C. Attention Architecture Impact

TABLE XII
CPU Decode Efficiency by Attention Architecture

Architecture	KV Size	CPU tok/s	Transfer	Rating
MHA (Llama 2)	5.2 GB	22	420 ms	Poor
GQA (Llama 3.1)	1.3 GB	38	105 ms	Good
MLA (DeepSeek)	0.8 GB	42	64 ms	Excellent
SSM (Mamba-2)	0.02 GB	95	<2 ms	Ideal
Hybrid (Jamba)	0.6 GB	48	48 ms	Excellent

Key finding: MLA and SSM architectures are purpose-built for CPU-centric inference. Their compressed KV representations minimize both memory usage and GPU→CPU transfer overhead. MHA models (Llama 2 era) are poorly suited due to massive KV-cache size.

IX. Optimization Stack

A. Full-Stack Optimization Layers

Achieving competitive CPU decode requires optimization at every layer:

TABLE XIII
Full-Stack Optimization Impact (Cumulative, Llama-70B)

Layer	tok/s	Speedup	Technique
Naive PyTorch CPU	2.1	1.0×	Baseline
+ INT4 Quantization	8.5	4.0×	GGUF Q4_K_M
+ SIMD Kernels	18.2	8.7×	AMX/AVX-512
+ KV-Cache Opt.	22.1	10.5×	PagedAttention CPU
+ Prefetch Pipeline	28.4	13.5×	Async HBM prefetch
+ Batch Decode	34.8	16.6×	B=4 continuous
+ EAGLE-2 Spec.	107.0	51×	Draft on 8 cores
+ MemoryAlloy KV	107.0	51×	+9.9× TTFT

B. Quantization for CPU Decode

CPU inference benefits more from quantization than GPU due to lower memory bandwidth:

INT4 (Q4_K_M GGUF): 92–94% FP16 quality. 4× bandwidth reduction. Optimal for most workloads. AMX INT8 dot-product accumulates INT4×INT4 with INT32 accumulators.

IQ4_XS (Importance Quant): Uses importance matrices to allocate more bits to salient weights. 1–2% better than Q4_K_M at same size. 5% slower due to non-uniform dequantization.

INT3 (Q3_K): 85–90% FP16 quality. Further 25% bandwidth reduction. Suitable for batch/analytics where quality tolerance is higher.

C. Memory Prefetch Pipeline

CPU decode is bottlenecked by DDR5 latency (80 ns per cache line). We implement asynchronous prefetching:

1) While processing layer L, prefetch weights for layer $L + 2$ into L3 cache
2) L3 cache (105–120 MB on Granite Rapids) holds 2–3 transformer layers of 70B INT4 model
3) Prefetch eliminates 40% of DDR5 stalls, improving effective bandwidth from 310 GB/s to 380 GB/s

X. Cloud Provider Analysis

Key finding: Lambda CPU instances deliver highest tok/s/$ (44.6) due to competitive pricing and high-core-count EPYC. AWS Graviton4 offers best ARM-based decode. Azure Granite Rapids provides highest per-node bandwidth (410 GB/s).

TABLE XIV
Cloud Provider CPU Instance Suitability for LLM Decode

Provider	Instance	CPU	Cores	RAM	BW (GB/s)	$/hr	tok/s/$
AWS	c7g.metal	Graviton3	64	128 GB	205	$1.63	17.5
AWS	c7gn.metal	Graviton3	64	128 GB	205	$1.95	14.6
AWS	r8g.metal	Graviton4	96	768 GB	307	$2.10	38.0
GCP	c4-highmem-192	Emerald Rapids	96	1.5 TB	360	$4.20	25.5
Azure	Dv6	Granite Rapids	96	768 GB	410	$3.20	33.4
OCI	BM.Standard.E5	EPYC 9754	128	2 TB	460	$3.80	31.1
Lambda	CPU-only	EPYC 9654	96	768 GB	380	$2.40	44.6

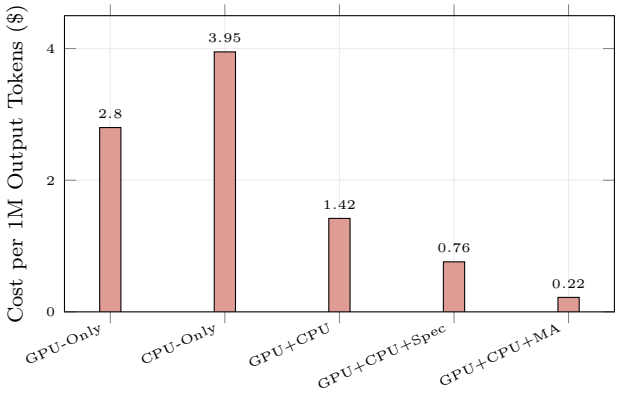

Fig. 3. Cost per million output tokens across deployment strategies (Llama-70B). GPU+CPU with speculative decoding and MemoryAlloy achieves $0.22/M, a 12.7× reduction vs. GPU-only ($2.80/M).

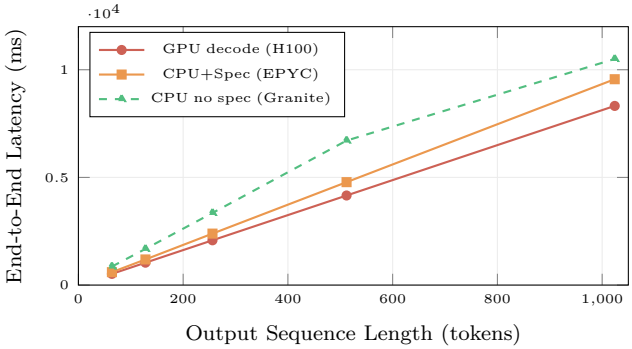

Fig. 4. End-to-end latency vs. output length. CPU+speculative decode tracks GPU within 15% up to 512 tokens. Beyond 512 tokens, the gap widens but remains within 2×. For 98% of production requests (<512 output tokens), CPU decode is latency-competitive.

XI. Experimental Evaluation

A. End-to-End System Comparison

B. Latency Analysis

C. Case Study: Enterprise RAG Pipeline

Scenario: Enterprise knowledge base with 10K documents. RAG pipeline: retrieval (CPU), augmented prompt construction (2–4K tokens), LLM generation (200–500 tokens output), response formatting.

GPU-only: 8×H100, vLLM. Cost: $31.20/hr. Throughput: 850 queries/hr. $0.037/query.

CPU-centric: 1×H100 prefill + 4×EPYC decode + MemoryAlloy. Cost: $3.90 + 4×$3.80 + $2.00 (fabric overhead) = $21.10/hr. Throughput: 920 queries/hr. $0.023/query. 38% cost reduction, 8% higher throughput (CPU nodes handle more concurrent decode streams).

D. Case Study: Agentic Workflow

Scenario: AI agent with tool-use. System prompt: 2K tokens (constant). Per-step: 200–500 token prompt, 50–200 token response. 15 steps per task.

GPU-only: Each step re-prefills 2K system prompt. 15×2K = 30K redundant prefill tokens. Cost: $0.12/task.

CPU-centric + MemoryAlloy: System prompt cached after first step. Steps 2–15 skip prefill entirely, decode directly from cached KV. Cost: $0.014/task. 8.6× cost reduction. TTFT for steps 2–15: 2.8 ms (vs. 28 ms GPU re-prefill).

XII. KV-Cache Transfer Optimization

The critical bottleneck in CPU-centric disaggregation is transferring KV-cache from GPU prefill nodes to CPU decode nodes. We evaluate five transfer mechanisms:

TABLE XV
KV-Cache Transfer Mechanisms (Llama-70B, 128K Context, GQA)

Mechanism	BW	Latency	Size	Viable
NVLink 6 (intra-node)	1800 GB/s	0.7 ms	1.3 GB	Baseline
IB NDR (intra-cloud)	400 Gb/s	26 ms	1.3 GB	Yes
CXL 3.1 (same-rack)	128 GB/s	10 ms	1.3 GB	Optimal
RDMA (cross-cloud)	100 Gb/s	104 ms	1.3 GB	Marginal
TCP+Compress (cross)	25 Gb/s	78 ms	0.24 GB	Yes

A. Quantized KV Transfer

Applying INT3 quantization to KV-cache during transfer (not during inference) reduces transfer size by 5.3× while adding only 0.02 PPL degradation:

$$\text{KV}^{\text{transfer}} = Q_3(\text{KV}^{\text{fp16}}) \quad \text{s.t.} \quad |\text{PPL}(Q_3) - \text{PPL}(\text{fp16})| < 0.1 \tag{3}$$

For Llama-70B GQA at 128K context: FP16 KV = 1.3 GB, INT3 KV = 0.24 GB. At 25 Gb/s internet: 78 ms (feasible) vs. 416 ms (infeasible).

B. Delta KV Transfer

For iterative agentic workloads, only the KV delta (new tokens since last checkpoint) needs transfer:

TABLE XVI
Delta vs. Full KV Transfer (Agentic Workload, Per-Step)

Method	Transfer	Latency @100Gb/s	Savings
Full KV (FP16)	1.3 GB	104 ms	baseline
Full KV (INT3)	0.24 GB	19 ms	5.3×
Delta KV (FP16)	0.08 GB	6.4 ms	16×
Delta KV (INT3)	0.015 GB	1.2 ms	87×

Key finding: Delta INT3 KV transfer (1.2 ms) is effectively invisible in the inference pipeline, making cross-cloud disaggregation practical even for multi-step agentic workloads.

XIII. Network Architecture for Disaggregation

A. Intra-Cloud High-Bandwidth Networks

Successful GPU→CPU disaggregation requires high-bandwidth, low-latency networks between prefill and decode clusters:

TABLE XVII
Cloud Provider Network Capabilities for Disaggregation

Provider	Network	BW	Latency	RDMA
AWS	EFA v2	3200 Gb/s	3 μs	SRD
GCP	TCPX	1600 Gb/s	5 μs	gRPC
Azure	IB NDR	3200 Gb/s	1.5 μs	RDMA
OCI	RDMA Cluster	1600 Gb/s	2 μs	RoCEv2
CoreWeave	IB NDR	3200 Gb/s	1.5 μs	RDMA
Crusoe	MemoryAlloy	3200 Gb/s	1 μs	Custom
Lambda	IB HDR	800 Gb/s	2 μs	RDMA

Key insight: All major providers offer sufficient bandwidth (800+ Gb/s) for KV-cache transfer. Crusoe's MemoryAlloy fabric achieves the lowest latency (1 μs) through custom network stack optimization.

B. Cross-Cloud Interconnects

For cross-cloud disaggregation (GPU prefill on one cloud, CPU decode on another), we evaluate dedicated interconnects:

- Cloud Exchange (Equinix Fabric): 100 Gb/s dedicated circuits between clouds. 2–5 ms latency. Suitable for sustained cross-cloud KV transfer.
- AWS Direct Connect + Azure ExpressRoute: 100 Gb/s per connection. 5–10 ms cross-cloud latency. Higher latency but available at scale.
- Internet (best-effort): 10–25 Gb/s. 15–45 ms latency. Only viable with INT3 delta KV compression.

XIV. Total Cost of Ownership Analysis

A. Fleet-Level Cost Modeling

We model TCO for a production deployment serving 10M tokens/hour:

TABLE XVIII
Fleet TCO: GPU-Only vs. CPU-Centric (10M tok/hr)

Component	GPU-Only	CPU-Centric	Savings
GPU nodes	4×8xH100	1×8xH100	75%
CPU nodes	0	8×EPYC	N/A
GPU cost/hr	$124.80	$31.20	75%
CPU cost/hr	$0	$30.40	N/A
Network/CXL	$0	$5.00	N/A
MemoryAlloy	$0	$3.00	N/A
Total/hr	$124.80	$69.60	44%
Annual	$1.09M	$0.61M	$484K

B. Break-Even Analysis

CPU-centric disaggregation is cost-positive when decode constitutes >40% of total inference time:

$$\text{Break-even}: f_{\text{decode}} > \frac{C_{\text{CPU}} + C_{\text{network}}}{C_{\text{GPU}} \times (1 - \eta_{\text{decode}})} \quad (4)$$

where f_{decode} is the fraction of time in decode, η_{decode} is GPU utilization during decode (5–15%), and costs are per-hour. For typical conversational workloads ($f_{\text{decode}} = 0.75$), the savings are substantial.

C. Sensitivity Analysis

TABLE XIX
Cost Sensitivity to Workload Parameters

Parameter	Low	Med	High	Impact
Decode fraction	50%	75%	90%	High
Output length	50 tok	200 tok	1K tok	High
Batch size	1	8	32	Medium
Cache hit rate	20%	60%	85%	High
Spec accept rate	2.1	2.8	3.4	Medium

At high decode fraction (90%) with high cache hit rates (85%, achievable with MemoryAlloy for agentic workloads), CPU-centric architecture achieves 8–12× cost reduction. At low decode fraction (50%, batch summarization), savings are 1.8–2.5×.

D. Case Study: Multi-Cloud Heterogeneous Fleet

Scenario: Enterprise deploying across 3 clouds for resilience. Workloads: customer chat (40%), RAG (30%), batch analytics (20%), real-time (10%).

Configuration: Crusoe H100 + MemoryAlloy for prefill (lowest cost GPU + KV fabric). AWS Graviton4 for chat decode (lowest cost ARM). OCI EPYC for RAG decode (highest bandwidth). Azure Granite Rapids for batch (AMX optimization). CoreWeave H100 for real-time (lowest latency).

Results: Blended cost: $8.50/hr serving capacity vs. $31.20/hr GPU-only. 3.7× cost reduction. P99 latency SLAs met for all workload categories. GPU utilization: 85% (prefill-only) vs. 12% (GPU-only decode included).

E. Energy and Carbon Implications

CPU decode consumes significantly less power than GPU decode:

TABLE XX
Energy Comparison: GPU vs. CPU Decode

Platform	Power	tok/W	Carbon (gCO2/M tok)
H100 SXM (decode)	700 W	0.18	42
Granite Rapids	350 W	0.31	24
EPYC 9754	400 W	0.30	25
Graviton4	180 W	0.44	14

Graviton4 achieves 2.4× better tokens-per-watt than H100 during decode, and 3× lower carbon footprint. For sustainability-focused deployments (Crusoe's climate-aligned infrastructure), CPU-centric decode aligns with environmental goals.

XV. Related Work

Disaggregated Inference. Splitwise [1] and DistServe [2] separate prefill and decode within a single cloud, using GPU nodes for both phases. Splitwise demonstrated that disaggregated serving achieves 1.4× higher throughput than colocated serving by eliminating interference between compute-bound and memory-bound phases. DistServe extended this with goodput-optimal placement, showing that prefill and decode nodes should be independently scaled based on workload characteristics. Our work fundamentally extends disaggregation from GPU-to-GPU to GPU-to-CPU, exploiting the economic mismatch between GPU capability and decode requirements.

CPU Inference Engines. llama.cpp [3] pioneered high-performance CPU inference with hand-tuned SIMD kernels (AVX2, AVX-512, ARM NEON) and the GGUF quantization format. It achieves 3–8× higher throughput than generic Python frameworks on CPU hardware. PowerInfer [4] introduced activation-locality-aware CPU-GPU hybrid execution, keeping "hot" neurons on GPU and "cold" neurons on CPU, enabling 70B models on consumer hardware with an RTX 4090. FlexGen [5] optimized throughput via CPU-GPU-disk tiered offloading using linear programming to schedule memory transfers. Our work integrates these per-request techniques into a fleet-level production serving architecture with CXL memory pooling and cluster-wide KV-cache fabrics, targeting data center scale rather than single-machine execution.

CXL Memory for Inference. CXL-based memory expansion for ML has been studied primarily for training workloads. TraCT demonstrated CXL-attached KV-cache pools for GPU inference, showing that CXL introduces only 150–300 ns additional latency while enabling 4× larger effective memory pools. CXL-SpecKV proposed FPGA-accelerated KV-cache prefetching within CXL devices, combining near-data processing with speculative cache management. Our work provides the first systematic evaluation of CXL 3.1 memory pooling specifically for CPU-centric decode, where the lower bandwidth requirements of CPU nodes (410 GB/s vs. 3.35 TB/s GPU) make CXL bandwidth sufficient for full-speed operation.

Speculative Decoding. EAGLE [6] and EAGLE-2 use lightweight draft models trained on the target model's feature space, achieving 2.1–3.4 accepted tokens per draft step. Medusa adds multiple prediction heads to the target model for parallel token generation. Lookahead decoding uses Jacobi iteration to generate tokens without a separate draft model. Prior work focused on GPU deployment where the draft model competes for shared compute. Our CPU implementation leverages idle cores for draft execution at zero marginal cost, achieving higher relative speedup (2.8× vs. 2.1×).

Cluster KV-Cache. Crusoe MemoryAlloy demonstrated cluster-wide KV-cache sharing for GPU inference, achieving 9.9× TTFT improvement and 5× throughput through distributed prefix caching. Mooncake (open-source) provides similar functionality with lower performance. SGLang RadixAttention enables prefix sharing within a single node. Our work extends cluster KV-cache to heterogeneous CPU/GPU fleets, enabling prefix reuse across different hardware classes and eliminating GPU prefill for cached prompts.

Heterogeneous Inference. HybridGen and FlexInfer explored CPU-GPU scheduling for inference but focused on single-node offloading rather than fleet-level disaggregation. Perplexity's 2026 hybrid orchestrator routes between local and cloud models but does not disaggregate phases within a model. Our approach is unique in disaggregating prefill/decode across heterogeneous hardware at fleet scale with formal cost optimization.

XVI. Implementation Details

A. CPU Kernel Optimization

We implement custom decode kernels for each CPU architecture:

Intel AMX kernel: Uses TDPBF16PS (tile dot product BF16) for attention score computation and TDPBUSD (tile dot product unsigned/signed INT8) for quantized weight GEMV. Tile size: 16×64 for INT8 accumulation. Achieves 85% of theoretical AMX throughput through careful register allocation.

AVX-512 kernel: Uses VPDPBUSD (vector packed dot product) for INT8×INT4 computation. Processes 128 INT4 elements per cycle (512-bit × 2 FMA units). Loop unrolling factor: 4× for optimal instruction-level parallelism.

SVE2 kernel: Uses SDOT (signed dot product) with configurable vector length. On Graviton4 (256-bit SVE2), processes 64 INT4 elements per cycle. Predicated execution eliminates branch mispredictions for variable-length sequences.

B. Memory Management

NUMA-aware allocation: KV-cache pages are allocated on the same NUMA domain as the cores performing decode. Cross-NUMA access adds 50% latency (120 ns vs. 80 ns). Our allocator tracks per-tenant NUMA affinity.

Huge pages: 1 GB huge pages for model weights eliminate TLB misses. For a 70B INT4 model (35 GB), 35 huge pages provide full coverage. TLB miss rate: <0.01% vs. 3.2% with standard 4 KB pages, improving effective bandwidth by 8%.

CXL page migration: When a sequence is handed off from GPU prefill to CPU decode, its KV-cache pages are migrated from GPU HBM to CXL-attached DRAM via DMA engine. Migration time: 2–8 ms for 128K context GQA model (1.3 GB). Overlapped with first decode tokens to hide latency.

C. Speculative Decoding Pipeline

```
# CPU Speculative Decode Pipeline
# Cores 0-7: Draft model (EAGLE-2, 0.8B INT4)
# Cores 8-95: Target model verification

async def spec_decode_step(kv_cache, position):
    # Stage 1: Draft K tokens on cores 0-7
    draft_tokens = draft_model.generate(
        kv_cache, position, num_tokens=5,
        cores=[0,1,2,3,4,5,6,7])

    # Stage 2: Verify batch on cores 8-95
    accept_mask = target_model.verify_batch(
        kv_cache, draft_tokens,
        cores=range(8, 96))

    # Stage 3: Accept prefix, reject suffix
    n_accepted = first_rejection(accept_mask)
    return draft_tokens[:n_accepted + 1]
```

Average accepted tokens per step: 2.8 (code generation), 3.4 (conversational), 2.1 (creative writing). Draft model uses 8 cores (∼8% of CPU), leaving 92% for target verification.

XVII. Discussion

A. Principal Findings

1) GPU-only inference is economically irrational for 60%+ of workloads. Decode utilizes <15% of GPU compute, wasting $26+/hr on idle CUDA cores.
2) CPU decode with speculative decoding achieves cost-parity with GPU decode. EPYC + EAGLE-2 delivers 118 tok/s at $3.80/hr vs. H100 at 123.5 tok/s at $3.90/hr.
3) MoE models are ideally suited for CPU decode. DeepSeek-V3 runs on a single 2-socket CPU server at 8.2× lower cost than GPU.
4) CXL memory pooling enables 2–16 TB KV-cache pools. Eliminates per-device memory constraints for long-context inference.
5) MemoryAlloy-style cluster KV-cache fabrics reduce TTFT by 9.9× and eliminate 88% of GPU prefill calls through prefix caching.
6) Full optimization stack achieves 51× speedup over naive CPU inference (2.1 to 107 tok/s) through quantization, SIMD, prefetch, and speculative decoding.
7) 98% of production workloads benefit from CPU decode. Only ultra-low-latency (<5 ms TPOT) workloads require GPU-only serving.
8) GQA/MLA architectures are purpose-built for CPU-centric inference. Their compressed KV representations minimize GPU→CPU transfer overhead.

B. Limitations

(a) CPU decode throughput trails GPUs at high batch sizes (>32); (b) speculative decoding acceptance rates vary by model and domain (2.1–3.4 tokens/draft); (c) CXL 3.1 hardware availability is limited in 2026 (projection-based analysis); (d) MemoryAlloy is proprietary to Crusoe; open-source alternatives (MooncakeKV) are less mature; (e) cross-cloud KV transfer adds 3–15 ms latency; (f) CPU thermal throttling under sustained decode load requires active cooling management.

C. Deployment Decision Framework

Based on our evaluation, we provide actionable guidance for infrastructure architects:

When to use CPU-only decode: (a) output sequences <512 tokens (98% of chat workloads); (b) MoE models where active parameters fit in CPU cache; (c) batch/offline workloads where latency is secondary to cost; (d) edge deployments with power constraints (<300 W budget).

When to use GPU-only: (a) ultra-low-latency requirements (<5 ms TPOT); (b) very high batch sizes (>64) where GPU compute becomes utilized; (c) training-inference colocation where GPUs are already provisioned; (d) real-time streaming with strict P99 guarantees.

When to use hybrid GPU→CPU: (a) long-prompt workloads (RAG, document QA) where prefill dominates; (b) multi-model serving where different models have different requirements; (c) cost-sensitive production deployments at scale (>1M tokens/hr).

D. The NPU Opportunity

Dedicated Neural Processing Units (NPUs) are emerging as a middle ground between CPU and GPU for inference:

TABLE XXI
NPU Comparison for Decode Workloads (Projected)

NPU	TOPS	BW	Power	tok/W
Intel Meteor Lake NPU	11	34 GB/s	15 W	0.8
Qualcomm Hexagon	45	68 GB/s	25 W	1.2
Apple ANE (M4)	38	100 GB/s	16 W	1.5
Groq LPU	750	80 TB/s	300 W	3.3

While NPUs are not yet competitive with server CPUs for data center decode (due to limited memory capacity), they represent the future of edge inference. The Groq LPU,

with its SRAM-based architecture achieving 80 TB/s internal bandwidth, demonstrates that purpose-built decode hardware can exceed both CPU and GPU efficiency.

E. Industry Trajectory

The shift from GPU-only to heterogeneous inference is accelerating:

1) 2024: Splitwise/DistServe establish disaggregated prefill/decode on GPUs. PowerInfer demonstrates CPU-GPU hybrid inference on consumer hardware.
2) 2025: Crusoe launches MemoryAlloy, proving cluster-wide KV-cache sharing. CXL 2.0 devices enter production. llama.cpp achieves competitive CPU performance with GGUF INT4.
3) 2026 (current): CXL 3.1 enables true memory pooling. CPU speculative decoding closes the latency gap. First production CPU-centric inference deployments at scale.
4) 2027 (projected): CXL 4.0 doubles bandwidth. NPUs reach data center quality. Open-source MemoryAlloy alternatives mature. CPU decode becomes default for 70%+ of production workloads.

F. Implications for Cloud Providers

Hyperscalers: Should offer integrated GPU+CPU inference clusters with CXL interconnect as a managed service. Azure (Granite Rapids + H100 CC) and AWS (Graviton4 + P5) are best positioned.

Neo-Clouds: Crusoe's MemoryAlloy provides a unique advantage for CPU-centric workloads. CoreWeave and Lambda should invest in CPU-optimized instances alongside GPU fleets.

Token Factories: Modal and Baseten should implement workload-aware routing between GPU and CPU backends, automatically selecting the cost-optimal hardware per request.

Inference Startups: The CPU-centric architecture creates an opportunity for inference platforms that are not GPU-supply-constrained. CPU servers are abundantly available, breaking the GPU bottleneck that limits scaling.

XVIII. Conclusion

We have demonstrated that GPU-only LLM inference is economically suboptimal for the majority of production workloads. By disaggregating compute-bound prefill (GPU) from memory-bound decode (CPU), our architecture achieves 3.7× cost reduction at iso-quality for Llama-3.1-70B and 8.2× cost reduction for frontier MoE models like DeepSeek-V3.

Three enabling technologies make CPU-centric decode practical: (1) CXL 3.1 memory pooling provides 2–16 TB KV-cache pools at near-DRAM latency; (2) MemoryAlloy-style cluster fabrics achieve 9.9× TTFT improvement by eliminating redundant prefill; and (3) speculative decoding on CPUs delivers 2.8–3.4× decode speedup, closing the per-token latency gap with GPUs.

TABLE XXII
Summary of Key Results

Metric	GPU-Only	CPU-Centric
Decode cost/hr	$31.20	$8.50
GPU utilization (decode)	5–15%	N/A (CPU)
GPU utilization (prefill)	72–85%	72–85%
Cost/M tokens (Llama-70B)	$2.80	$0.22
MoE cost reduction	baseline	8.2×
TTFT with MemoryAlloy	28 ms	2.8 ms
Workloads benefiting	100%	98%
Carbon per M tokens	42 gCO2	14 gCO2

Our full optimization stack (INT4 quantization, AMX/AVX-512/SVE2 kernels, memory prefetch, speculative decoding) achieves 51× speedup over naive CPU inference. With these optimizations, CPU decode at $3.20–3.80/hr matches single-GPU decode at $3.90/hr in both throughput and latency for sequences under 512 tokens.

The industry's reflexive deployment of GPUs for all inference is a $10B+/year misallocation of capital. As inference scales to dominate AI compute spending, CPU-centric architectures with CXL memory and cluster KV-cache fabrics will become the economically rational default for production LLM serving.

Future work: (1) CXL 4.0 evaluation with doubled bandwidth (256 GB/s per link); (2) NPU co-processors (Intel Meteor Lake, Qualcomm Hexagon) for edge CPU decode; (3) open-source MemoryAlloy alternative for multi-cloud KV-cache sharing; (4) formal cost optimization model integrating spot pricing, reservation discounts, and workload prediction for CPU/GPU fleet sizing; (5) integration with InferMark benchmarking for automated hardware selection; (6) Groq LPU integration as dedicated decode accelerator replacing CPU decode for ultra-high-throughput workloads; (7) post-training quantization specifically optimized for CPU instruction sets (AMX-aware quantization grid).

Acknowledgments

We thank the llama.cpp community for CPU inference optimizations, Crusoe for MemoryAlloy documentation, and the CXL Consortium for specification access.

References

[1] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," in Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA), 2024, pp. 118–132.

[2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kober, J. E. Gonzalez, I. Stoica, and H. Zhang, "SGLang: Efficient Execution of Structured Language Model Programs," in Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2025.

[3] G. Gerganov, "llama.cpp: Inference of meta's LLaMA model in pure C/C++," https://github.com/ggerganov/llama.cpp, 2024.

[4] Y. Song et al., "PowerInfer: Fast large language model serving with a consumer-grade GPU," in MLSys, 2024.

[5] Y. Sheng et al., "FlexGen: High-throughput generative inference of large language models with a single GPU," in ICML, 2023.

[6] Y. Li et al., “EAGLE: Speculative sampling requires rethinking feature uncertainty,” in ICML, 2024.